

บทที่ 2

J2ME

2.1 ความเป็นมาของ J2ME

ในระยะ 2 - 3 ปีที่ผ่านมา ตลาดของอุปกรณ์สื่อสารไร้สายได้มีการขยายตัวอย่างต่อเนื่องและมีแนวโน้มจะเติบโตยิ่งขึ้นไปอีกเมื่อมีการนำระบบเครือข่ายอุปกรณ์สื่อสารไร้สายรุ่นที่ 3 (3rd generation – 3G) เข้ามาใช้งาน ซึ่งเป็นระบบสื่อสารไร้สายที่รับส่งข้อมูลโดยใช้แบนด์วิดท์สูงขึ้นคือ 384 กิโลบิต ต่อวินาที ขณะที่อุปกรณ์ไร้สายรุ่นที่ 2 มีแบนด์วิดท์เพียง 19.2 กิโลบิตต่อวินาที

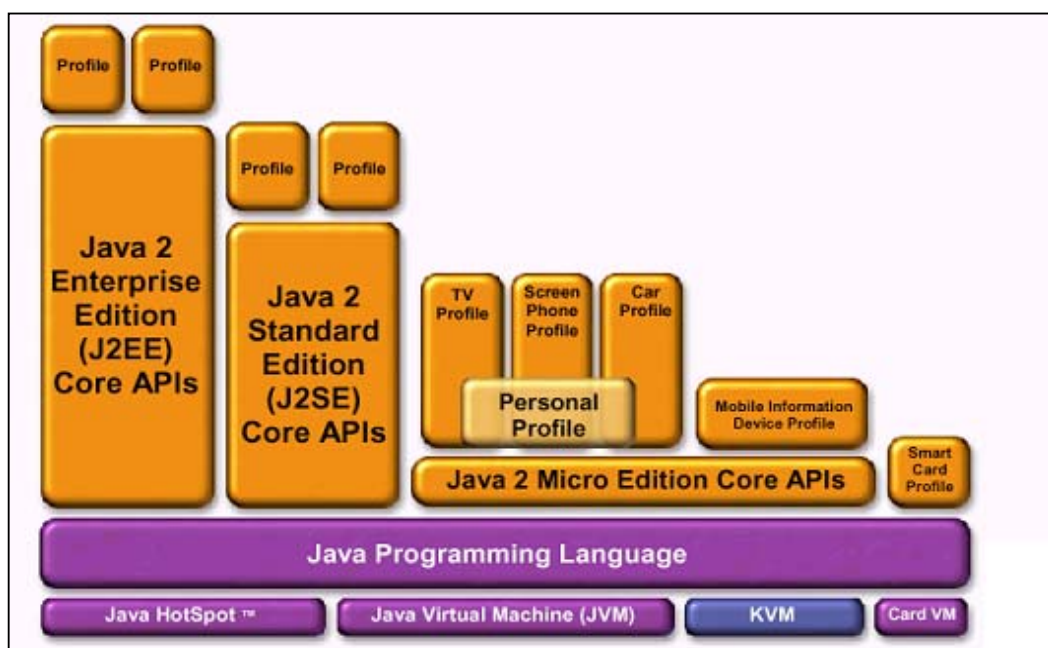
ช่องทางการสื่อสารที่เพิ่มขึ้นนี้ทำให้สามารถรองรับปริมาณการใช้โทรศัพท์ได้มากขึ้นและให้บริการข้อมูลภาพและเสียงคุณภาพสูงได้อีกด้วย จากระบบสื่อสารไร้สายรุ่นที่ 3 นี้ทำให้แอปพลิเคชัน อินเทอร์เน็ต บนอุปกรณ์สื่อสารไร้สายมีการพัฒนามากขึ้น อีกทั้งทุกวันนี้ผู้คนนิยมรับข่าวสาร เช่น ข่าว สภาพอากาศ จดหมายอิเล็กทรอนิกส์ (email) โดยตรงจากอินเทอร์เน็ตผ่านโทรศัพท์เคลื่อนที่, พีดีเอ, เพจเจอร์ ซึ่งแอปพลิเคชันที่มีความสามารถเหล่านี้ ส่วนใหญ่ใช้เทคโนโลยี เช่น โพรโทคอลสำหรับอุปกรณ์ไร้สาย (Wireless Application Protocol-WAP) และ ไอโมด (i-mode) แต่เทคโนโลยีดังกล่าวมีข้อจำกัดบางประการ ซึ่งรวมถึงการขาดระบบรักษาความปลอดภัยที่ดี, ส่วนติดต่อกับผู้ใช้งาน (user interface) ที่ยังไม่ดีเท่าที่ควร และข้อจำกัดของระยะเวลาการต่ออินเทอร์เน็ต

ดังนั้นทางบริษัทซันไมโครซิสเต็มได้ออกแบบ Java 2 Platform, Micro Edition (J2ME) ซึ่งเป็นหนทางหนึ่งในการแก้ไขข้อจำกัดดังกล่าว และสามารถทำงานที่ซับซ้อนได้มากขึ้น เช่น แอปพลิเคชันไคลเอนต์/เซิร์ฟเวอร์ (client/server) สำหรับองค์กร, เกมส์, ฟังเพลง เนื่องจากอุปกรณ์สื่อสารไร้สายเป็นอุปกรณ์ขนาดเล็กทำให้มีข้อจำกัดหลายด้านเมื่อเทียบกับคอมพิวเตอร์ เช่น ขนาดของหน่วยความจำ, ความสามารถในการประมวลผล แต่ด้วยเทคโนโลยีของ J2ME ซึ่งออกแบบโดยเน้นถึงกลุ่มผู้ใช้อุปกรณ์อิเล็กทรอนิกส์ และอุปกรณ์ฝังตัว (Embedded) รวมถึงอุปกรณ์สื่อสารไร้สาย ทำให้นักพัฒนาสามารถพัฒนาแอปพลิเคชันสำหรับอุปกรณ์เหล่านั้นได้

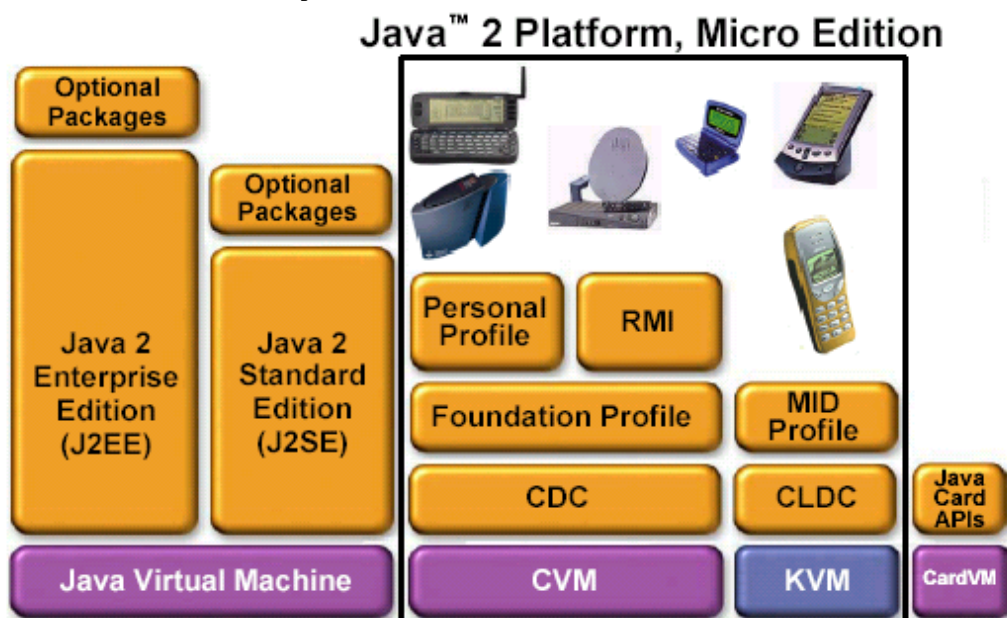
ทางบริษัทซันไมโครซิสเต็มได้จัดกลุ่มของเทคโนโลยีของจาวาเป็น 3 รุ่น ดังรูปที่ 2-1 คือ

- Java 2 Platform, Enterprise Edition (J2EE) สร้างบน J2SE โดยเพิ่ม Java Enterprise API เป็นแพลตฟอร์มสำหรับพัฒนา enterprise-class ที่ต้องดีพลอยบนเซิร์ฟเวอร์
- Java 2 Platform, Standard Edition (J2SE) เป็นแพลตฟอร์มจาวาที่เหมาะสมสำหรับการทำงานบนเครื่องคอมพิวเตอร์พีซีประกอบด้วยบริการมาตรฐานต่างๆ ของจาวาสำหรับ applet และแอปพลิเคชัน รู้จักในชื่อ Java Development Kit (JDK)

- Java 2 Platform , Micro Edition (J2ME) เป็นเทคโนโลยีจาวาที่ออกแบบมาเพื่อตอบสนองความต้องการของผู้ใช้อุปกรณ์อิเล็กทรอนิกส์และอุปกรณ์ฝังตัวที่มีขนาดของหน่วยความจำ , ความสามารถในการประมวลผลที่จำกัด และความสามารถในการทำงานของส่วนติดต่อกราฟิกกับผู้ใช้ (GUI) โดยเฉพาะเช่น โทรศัพท์เคลื่อนที่ , พีดีเอ , เพจเจอร์ เป็นต้น ดังรูปที่ 2-2



รูปที่ 2-1 โครงสร้างของเทคโนโลยีจาวา



รูปที่ 2-2 แสดงเทคโนโลยี Java 2 Platform Micro Edition

จากรูปที่ 2-1 และ 2-2 จะเห็นว่าจาวาเทคโนโลยีแต่ละแพลตฟอร์มจะมีเวอร์ชวลแมชชีนเฉพาะซึ่งปรับแต่งเป็นพิเศษ เพื่อสนับสนุนการทำงานของแอปพลิเคชันที่จะนำไปใช้ได้ดียิ่งขึ้น HotSpot VM เป็นเวอร์ชวลแมชชีนของจาวาที่ปรับแต่งมาเพื่อเพิ่มประสิทธิภาพของแอปพลิเคชันบนเครื่องเซิร์ฟเวอร์ที่เขียนด้วย J2EE และยังสามารถทำงานเข้ากับแอปพลิเคชันที่เขียนด้วย J2SE ได้อีกด้วย JVM (Java Virtual Machine) เป็นเวอร์ชวลแมชชีนสำหรับจาวาทั่วไปที่ปรับแต่งเพื่อการใช้งานกับแอปพลิเคชันบนเครื่องคอมพิวเตอร์พีซี

เวอร์ชวลแมชชีนที่ออกแบบมาเพื่อสำหรับใช้งานกับ J2ME มี 2 ประเภท คือ กิโลไบต์เวอร์ชวลแมชชีน (KVM) และ คอมแพ็คเวอร์ชวลแมชชีน (CVM) ซึ่งมีขนาดเล็กและใช้ทรัพยากรของระบบน้อยกว่าเวอร์ชวลแมชชีน 2 ประเภทก่อนหน้านี้

ประโยชน์ของ J2ME ในการพัฒนาแอปพลิเคชัน

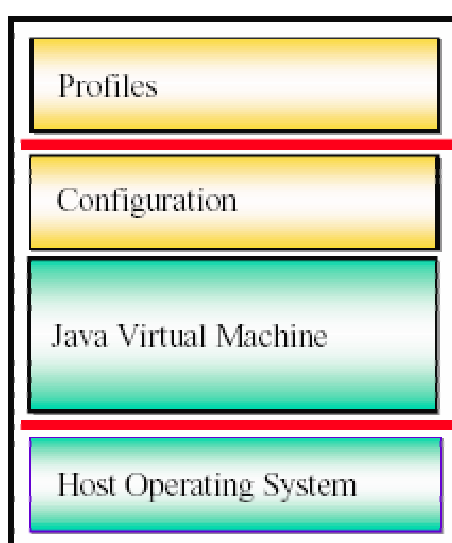
- เป็นแพลตฟอร์มอิสระ แอปพลิเคชันบนอุปกรณ์ไร้สายที่พัฒนาด้วยจาวาสามารถทำงานบนอุปกรณ์ที่ผลิตโดยผู้ผลิตต่างกันได้ ช่วยให้เคลื่อนย้ายแอปพลิเคชันไปใช้งานได้สะดวกยิ่งขึ้น
- แอปพลิเคชันบนอุปกรณ์ไร้สายมักทำงานเกี่ยวกับระบบเครือข่าย เพื่อให้ผู้ใช้งานสามารถติดต่อสื่อสารได้ จาวาจึงได้ออกแบบให้มีความสามารถด้านเครือข่ายในตัวเอง โดยมีคลาสไลบรารีระบบเครือข่ายให้ทำการเขียนแอปพลิเคชันด้านเครือข่ายง่ายขึ้น
- มีระบบรักษาความปลอดภัยหลายระดับตั้งแต่ คลาสโหลดเดอร์ , ไบต์โค้ดเวอร์ไฟเลอร์ ไปจนถึงโปรแกรมรักษาความปลอดภัย เช่น ตัวจัดการความปลอดภัย (Security Manager) ซึ่งสามารถป้องกันไคลเอ็นต์จากแอปพลิเคชันที่น่าไว้วางใจ นอกจากนี้ยังมี APIs รักษาความปลอดภัยการรับส่งข้อมูลผ่านเว็บอีกด้วย
- มีระบบกลไกการติดตั้งแอปพลิเคชันซึ่งยินยอมให้ดาวน์โหลดแอปพลิเคชันผ่านระบบสื่อสารไร้สายแล้วนำมาติดตั้งที่อุปกรณ์ กลไกนี้ช่วยให้ผู้ใช้งานสามารถดาวน์โหลดและปรับแต่งแอปพลิเคชันบนอุปกรณ์ได้ตามต้องการ
- มีส่วนติดต่อกับผู้ใช้ที่หลากหลาย และมีคลาสไลบรารีรองรับอีเวนต์ต่างๆ โดยใช้พื้นที่การแสดงผลบนหน้าจออุปกรณ์ไร้สายให้เกิดประโยชน์สูงสุด ส่วนติดต่อกับผู้ใช้นี้ทำให้อุปกรณ์ไร้สายสามารถสนับสนุนการทำงานของวิดีโอเกมสลับสูง และแอปพลิเคชันที่ซับซ้อนได้

2.2 สถาปัตยกรรมของ J2ME

J2ME เป็นแพลตฟอร์มจาวาที่ออกแบบมาเพื่อใช้งานกับแอปพลิเคชันที่ทำงานบนอุปกรณ์ขนาดเล็ก เช่น โทรศัพท์มือถือ , พีดีเอ , โทรศัพท์พร้อมจอภาพที่เชื่อมต่อกับอินเทอร์เน็ต , โทรศัพท์ดิจิทัลขนาดเล็ก , อุปกรณ์บันทึกและระบบนำทางในรถยนต์

J2ME เป็นแพลตฟอร์มที่ถูกกำหนดไว้ด้วยส่วนประกอบหลายส่วน เพื่อช่วยให้สามารถตอบสนองความต้องการที่หลากหลายของอุปกรณ์ที่มีอยู่มากมายได้ สถาปัตยกรรมของ J2ME ถูกออกแบบมาให้มีลักษณะเป็นส่วนๆ (Modular) และแต่ละส่วนนั้นสามารถขยายหรือปรับขนาดได้ (Scalable) ซึ่งประกอบไปด้วย 3 เลเยอร์ บน Host Operating System ของอุปกรณ์ ดังรูปที่ 2-3 คือ

- โพรไฟล์
- คอนฟิกูเรชัน
- จาวาเวอร์ชวลแมชีน



รูปที่ 2-3 สถาปัตยกรรมของ J2ME

2.2.1 จาวาเวอร์ชวลแมชีน

เป็นเวอร์ชวลแมชีนที่นำจาวาเวอร์ชวลแมชีน มาปรับแต่งให้เข้ากับระบบปฏิบัติการของอุปกรณ์ และรองรับแต่ละคอนฟิกูเรชันของ J2ME สิ่งทำงานร่วมกับเวอร์ชวลแมชีนคือ คอนฟิกูเรชันและโพรไฟล์ ซึ่งจัดหาคلاس APIs สำหรับเอนไวรอนเมนต์ แต่ละคอนฟิกูเรชันของ J2ME

2.2.2 คอนฟิกูเรชัน

ประกอบด้วยชุดคลาสไลบรารีขั้นต่ำของจาวาเวอร์ชวลแมชีน และจาวาคلاسสำหรับอุปกรณ์แต่ละประเภท หรืออาจกล่าวได้ว่าคอนฟิกูเรชันเป็นตัวกำหนดคุณสมบัติหรือไลบรารีขั้นต่ำของแพลตฟอร์ม จาวาที่นักพัฒนาคาดว่าจะต้องมีในทุกอุปกรณ์ เพื่อเขียนโปรแกรมจาวาสำหรับรองรับความต้องการขั้นพื้นฐานใน

การทำงานบนอุปกรณ์ขนาดเล็ก ซึ่งอุปกรณ์แต่ละตัวจะใช้จาวาเวอร์ชวลแมชีนในการเขียนโปรแกรมที่ต่างกัน ดังนั้นจึงแบ่งคอนฟิกูเรชันของ J2ME ออกเป็น 2 ชนิด

- 1) Connected Limited Device Configuration (CLDC) เป็นคอนฟิกูเรชันของอุปกรณ์ที่มีลักษณะดังต่อไปนี้
 - มีหน่วยความจำ (ROM) ขั้นต่ำ 128 กิโลไบต์ เพื่อใช้ในการเก็บไลบรารีที่จำเป็นในการทำงาน
 - มีหน่วยความจำ (RAM) ขั้นต่ำ 32 กิโลไบต์ ใช้สำหรับจองพื้นที่หน่วยความจำขณะรันไทม์
 - ความเร็วของหน่วยประมวลผล (CPU) อย่างน้อยเริ่มต้นที่ 8-32 เมกะเฮิร์ต
 - มีหน่วยประมวลผลขนาด 16 หรือ 32 บิต
 - มีส่วนติดต่อกับผู้ใช้จำกัด
 - มีแหล่งจ่ายพลังงานจำกัด ส่วนมากทำงานโดยใช้พลังงานจากแบตเตอรี่
 - การเชื่อมต่อกับเครือข่ายอยู่ในรูปแบบไร้สาย โดยมีแบนด์วิธจำกัดและมีการส่งข้อมูลไม่ต่อเนื่อง ความเร็ว 9,600 บิตต่อวินาที หรือน้อยกว่า

เวอร์ชวลแมชีนใน CLDC เรียกว่า KVM (Kilobyte Virtual Machine) เป็นซอฟต์แวร์ที่ทำหน้าที่เป็นตัวกลางในการแปลโปรแกรมที่เขียนขึ้นในภาษาจาวาไปเป็นภาษาเครื่อง (machine code) เพื่อส่งต่อไปให้กับระบบปฏิบัติการของอุปกรณ์ดำเนินการต่อไป และเป็นไปในลักษณะเดียวกัน โดย KVM นี้เหมาะสำหรับการใช้งานร่วมกับอุปกรณ์อิเล็กทรอนิกส์ที่มีขนาดหน่วยความจำในช่วง 128-512 กิโลไบต์ จัดได้ว่าเล็กมากเมื่อเทียบกับเวอร์ชวลแมชีนของ J2SE ที่มีขนาด 32 เมกะไบต์ เนื่องจากหน่วยความจำที่มีขนาดเล็กจึงทำให้จาวาเวอร์ชวลแมชีนมีความแตกต่างไปจากจาวาเวอร์ชวลแมชีนแบบอื่นๆ ดังนี้

- การไม่สนับสนุนชนิดของข้อมูลแบบเลขทศนิยม (Float) และ ตัวเลขขนาด 32 บิต (Double) เนื่องจาก CLDC ไม่ได้รวมความสามารถในการสนับสนุนตัวเลขทศนิยมเข้าไว้ ดังนั้นจึงไม่สามารถใช้ข้อมูลพื้นฐานประเภทเลขทศนิยม และตัวเลขขนาด 32 บิต รวมทั้งไม่สามารถนำคลาส 2 ชนิด คือ `java.lang.Float` และ `java.lang.Double` มาใช้งานได้ หากจำเป็นที่จะต้องใช้การคำนวณทศนิยมในโปรแกรมที่พัฒนาด้วย J2ME อาจทำได้โดยใช้ซอฟต์แวร์ เช่น MathFP เป็นตัวจำลองการคำนวณทศนิยมด้วยจำนวนเต็มคงที่
- การไม่สนับสนุน Java Native Interface (JNI) เนื่องจากสาเหตุ 2 ประการ คือ ในแบบจำลองความปลอดภัยของ CLDC ผู้เขียนแอปพลิเคชันไม่สามารถโหลดไลบรารีใหม่ๆ ที่ยังมีฟังก์ชันเดิมของอุปกรณ์อยู่ และไม่สามารถเข้าถึงฟังก์ชันเดิมของอุปกรณ์ที่ไม่มีไลบรารีของจาวาได้ เพราะว่าการนำ JNI มาใช้เป็นการเพิ่มภาระให้กับหน่วยความจำบนอุปกรณ์ CLDC
- การไม่มีรูปแบบ Reflection ทำให้โปรแกรม CLDC ไม่สามารถตรวจสอบเนื้อหาในคลาสออบเจกต์เมธอดได้ รวมถึงการเรียกใช้คำสั่งของออบเจกต์ที่ไม่ได้อยู่ที่เดียวกัน (Remote Method Invocation - RMI) และการเรียงลำดับออบเจกต์ (Object Serialization)

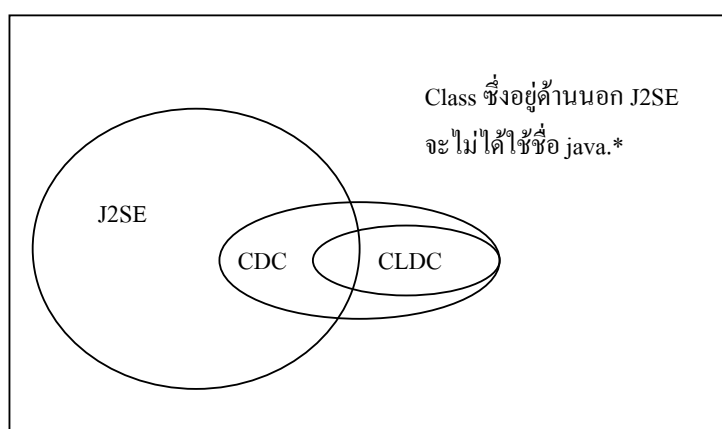
- การไม่มี user-defined , Java-level class loaders เนื่องจากเหตุผลด้านความปลอดภัยและโปรแกรมบรรจุคลาสใน KVM ก็ไม่อนุญาตให้ผู้ใช้งานลบล้าง เขียนทับหรือกำหนดค่าใหม่ได้
- การไม่สนับสนุนเธรดออบเจกต์ หรือ daemon threads KVM จะสนับสนุนการทำงานแบบ Multi-Thread แต่จะไม่รองรับเธรดออบเจกต์ หรือ daemon thread เราจะต้องใช้การเก็บสะสมออบเจกต์ (Collection object) ในการเก็บเธรดออบเจกต์ เพื่อทำงานแบบกลุ่มได้
- การไม่สนับสนุน finalization ของ class instances Finalization คือ กระบวนการที่ให้ออบเจกต์ล้างตัวเองก่อนที่จะถูกเก็บกวาด ซึ่งไลบรารีของ CLDC ไม่ได้รวมเอาเมธอด finalize ไว้ในคลาสออบเจกต์ เพื่อให้ KVM เก็บกวาดได้สะดวกขึ้น
- การไม่สนับสนุน weak reference ทำให้โปรแกรมไม่ได้รับแจ้งเมื่อมีการตรวจสอบพบว่าสามารถดึงออบเจกต์ที่ต้องการคืนมาจากออบเจกต์อื่นได้
- มีข้อจำกัดของการจัดการเกี่ยวกับข้อผิดพลาด (error handling) CLDC รองรับความผิดพลาด ในการทำงานในวงจำกัด โดยกำหนดคลาสแสดงข้อผิดพลาดไว้เพียง 2 คลาส คือ java.lang.VirtualMachineError และ java.lang.OutOfMemoryError โดยได้นำคลาสส่วนใหญ่ออกไป เนื่องจาก
 - ในระบบอุปกรณ์ฝังตัว การคืนสู่สภาวะปกติหลังจากเกิดภาวะที่ทำงานผิดพลาดเป็นความสามารถเฉพาะของบางอุปกรณ์เท่านั้น
 - ความผิดปกติประเภทนี้มักไม่สามารถแก้ไขได้ เพราะฉะนั้นการบรรจุความสามารถในการจัดการข้อผิดพลาดอย่างเต็มรูปแบบทำให้ต้องเสียค่าใช้จ่ายมากและยังต้องจัดลำดับความสำคัญไว้สูง ทั้งที่อุปกรณ์ CLDC มีทรัพยากรจำกัด

คลาสไลบรารีต่างๆ จะสืบทอดมาจากคลาสจาวาของ J2SE ดังรูปที่ 2-4 แต่จะคัดบางคลาสออกไป ดังนั้นใน CLDC จะเหลืออยู่ 4 แพ็คเกจ คือ java.lang , java.util , java.io , javax.microedition ซึ่งเป็นแพ็คเกจที่เพิ่มขึ้นมาใน J2ME เหตุผลในการกำจัดคลาสบางคลาสออกไปเกี่ยวข้องกับหลักการเบื้องต้น 2 อย่างของ CLDC ประการแรกคือ CLDC ต้องมีขนาดไม่เกิน 512 Kbytes ประการที่สองคือ คอนฟิгурชัน จะตอบสนองความต้องการพื้นฐานของอุปกรณ์ แต่ในแต่ละอุปกรณ์จะมีระบบแตกต่างกันซึ่งไม่สามารถสนับสนุนคลาสใน J2ME ทั้งหมดได้ ดังนั้นจึงนำคลาสที่ไม่สามารถนำมาใช้งานได้ออกไป

- 2) Connected Device Configuration (CDC) เป็นคอนฟิгурชันของอุปกรณ์ที่มีลักษณะดังต่อไปนี้
 - มีหน่วยความจำ (ROM) ขั้นต่ำ 512 กิโลไบต์ เพื่อใช้ในการเก็บไลบรารีที่จำเป็นในการทำงาน
 - มีหน่วยความจำ (RAM) ขั้นต่ำ 256 กิโลไบต์ ใช้สำหรับจองพื้นที่หน่วยความจำขณะรันไทม์
 - มีหน่วยประมวลผลขนาด 32 บิต
 - การเชื่อมต่อกับเน็ตเวิร์คเป็นไปอย่างต่อเนื่องและมีแบนด์วิธสูง

เวอร์ชวลแมชีนของ CDC เรียกว่า CVM (Compact Virtual Machine) คือ เวอร์ชวลแมชีนสำหรับ Foundation Profile ซึ่ง CVM นี้ถูกสร้างด้วยภาษา C และ Assembly CDC ใช้สำหรับอุปกรณ์ที่มีความสามารถน้อยกว่าเครื่องคอมพิวเตอร์และมีหน่วยความจำมากกว่า 512 กิโลไบต์ อุปกรณ์เหล่านั้นได้แก่ เซตท็อปบล็อกซ์, ระบบนำทางและระบบบันเทิงในรถยนต์, ระบบโทรศัพท์ผ่านอินเทอร์เน็ต เป็นต้น

จาวาแพ็คเกจที่ใช้ใน CDC คือ java.lang , java.util , java.io , java.net , java.text และ java.security



รูปที่ 2-4 แสดงความสัมพันธ์ของคลาสระหว่าง J2SE , CDC และ CLDC

2.2.3 โพรไฟล์

โพรไฟล์ทำหน้าที่เสริมคอนฟิгурชันซึ่งประกอบด้วยคลาสไลบรารีพื้นฐาน โดยจะรองรับความต้องการของอุปกรณ์ที่มีลักษณะเฉพาะหรือแอปพลิเคชันเฉพาะ โดยโพรไฟล์จะทำงานไม่ได้ถ้าปราศจากคอนฟิгурชัน และเวอร์ชวลแมชีนที่จัดหาคلاس และ API หลัก ปกติแล้วโพรไฟล์จะจัดหาส่วนติดต่อกับผู้ใช้, วิธีการอินพุต, การเชื่อมต่อเครือข่าย และกลไกการรักษาข้อมูลให้คงอยู่ดั้งเดิม (Data persistent mechanism) สำหรับแต่ละอุปกรณ์ ซึ่งโพรไฟล์จะช่วยสร้างความสมบูรณ์ให้กับเครื่องมือที่ช่วยในการพัฒนาแอปพลิเคชัน (Toolkit) บนอุปกรณ์ที่ต้องการ

ในปัจจุบันมีการระบุโพรไฟล์ไว้ 5 แบบ ดังตารางที่ 2-1 ซึ่งโพรไฟล์แต่ละตัวทำหน้าที่เสริมการทำงานของคอนฟิгурชันดังนี้

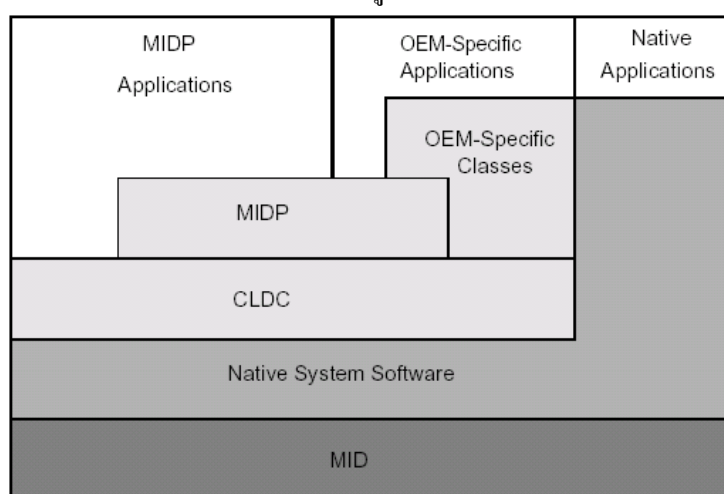
โพรไฟล์	วัตถุประสงค์	คอนฟิгурชัน
Mobile Information Device Profile (MIDP)	เตรียมส่วนติดต่อกับผู้ใช้, หน่วยเก็บข้อมูลแบบคงตัว (persistent storage), ความสามารถด้านเครือข่ายไว้ให้อุปกรณ์ไร้สาย เช่น โทรศัพท์มือถือ	CLDC

Personal Data Assistant (PDA)	มีส่วนติดต่อกับผู้ใช้และ APIs ด้านการเก็บข้อมูล สำหรับอุปกรณ์พกพาขนาดเล็กที่ใช้ระบบ palmOS เช่น Palm , Handspring	CLDC
Foundation	ใช้งานกับอุปกรณ์ที่ต้องติดตั้งจาวาเวอร์ชวลแมชีน เป็นไพรไฟล์ มาตรฐานสำหรับอุปกรณ์ CDC ทุกตัวที่ไม่มีความสามารถติดต่อกราฟิกกับผู้ใช้	CDC
Personal	เป็นส่วนเสริม Foundation Profile เพื่อทำงานกับ แอปพลิเคชัน ที่พัฒนาสำหรับ PersonalJava Application Environment Specification รุ่น 1.1.x และ 1.2.x และจัดหาความสามารถในการเชื่อมต่อเครือข่าย รวมไปถึงความสามารถด้านการติดต่อกราฟิกกับผู้ใช้ ของการเรียกใช้ Applet	CDC
Remote Method Invocation (RMI)	ส่วนเสริม Foundation Profile เพื่อจัดหา RMI ระหว่างจาวากับจาวา , สามารถทำงานร่วมกับแอปพลิเคชันระบบอื่นได้ และจัดหาความสามารถในการเชื่อมต่อเครือข่ายที่ดีกว่า Personal ไพรไฟล์	CDC

ตารางที่ 2-1 แสดงไพรไฟล์แบบต่างๆ

2.3 สถาปัตยกรรมของ MIDP

จากที่กล่าวมาแล้วข้างต้น MIDP, KVM และ CLDC ได้กลายมาเป็นรากฐานในการพัฒนาจาวา แอปพลิเคชันบนอุปกรณ์สื่อสารไร้สาย ซึ่งมีโครงสร้างตามรูปที่ 2-5



รูปที่ 2-5 โครงสร้างของ MIDP

จากรูปที่ 2-5 แบ่งโครงสร้างของ MIDP ออกเป็นเลเยอร์ต่างๆ ดังนี้

- **MID (Mobile Information Device)** เป็นเลเยอร์ของฮาร์ดแวร์ ซึ่งหมายถึง อุปกรณ์ประเภท โทรศัพท์ หรือ เพจเจอร์รับส่งข้อความ (เช่น RIM รุ่น Blackberry 950)
- **Native System Software** เป็นเลเยอร์ซอฟต์แวร์ของระบบที่ติดตั้งมากับอุปกรณ์ หมายถึง ส่วนที่เป็นระบบปฏิบัติการและไลบรารีของระบบที่บริษัทผู้ผลิตอุปกรณ์ให้มา
- **Native Applications** เป็นเลเยอร์ในส่วนของแอปพลิเคชันพื้นฐานที่ผู้ผลิตจะใส่ไว้ในตัวอุปกรณ์ เช่น การปรับเสียงเรียกเข้าของโทรศัพท์มือถือ , การปรับระดับความดังของเสียง
- **CLDC** เป็นเลเยอร์ที่เตรียม APIs หลักของจาวาให้แอปพลิเคชันบนอุปกรณ์ไร้สาย ซึ่งเป็นส่วนของKVM และไลบรารีที่ถูกกำหนดโดย CLDC
- **MIDP** เป็นเลเยอร์ของโพรไฟล์ที่จัดเตรียมคลาสไลบรารีสำหรับส่วนติดต่อกราฟิกกับ ผู้ใช้ และหน่วยเก็บข้อมูลแบบคงตัว รวมถึงระบบเครือข่ายด้วย
- **MIDP Applications** เป็นส่วนของ MIDlet ที่ใช้ไลบรารีที่มีอยู่ใน MIDP และ CLDC
- **OEM-Specific (Original Equipment Manufacturer) Classes** เป็นเลเยอร์ของคลาสไลบรารีที่ผู้ผลิตกำหนดขึ้นตามคุณลักษณะเฉพาะของแต่ละอุปกรณ์ เช่น ความสามารถในการตอบรับโทรศัพท์อัตโนมัติ , การเข้าถึงข้อมูลภายในสมุดโทรศัพท์
- **OEM-Specific Applications** เป็นเลเยอร์ของแอปพลิเคชันที่ใช้ OEM-Specific Classes

2.3.1 แอปพลิเคชันบนอุปกรณ์ไร้สาย MIDlet

MIDlet คือ แอปพลิเคชันที่เขียนโดยใช้ APIs ของ MIDP มีลักษณะการทำงานคล้ายจาวา แอปเพล็ต (Java Applet) คือ แอปพลิเคชันจะทำงานบนเวอร์ชวลแมชีน และถูกควบคุมการทำงานโดยแอปพลิเคชันอีกตัวหนึ่ง ซึ่งสำหรับแอปเพล็ต (Applet) จะถูกควบคุมการทำงานโดยเว็บเบราว์เซอร์ (Web Browser) หรือแอปเพล็ตวิวเวอร์ (Applet Viewer) และ MIDlet จะถูกควบคุมการทำงานโดยซอฟต์แวร์จัดการแอปพลิเคชัน (Application Management Software - AMS)

ซอฟต์แวร์จัดการแอปพลิเคชันจะไปจัดการควบคุมการทำงานของ MIDlet ทั้งหมดทุกขั้นตอน ดังนี้

1. Retrieval (Load)

ซอฟต์แวร์จัดการแอปพลิเคชันจะทำหน้าที่โหลด MIDlet เข้าสู่หน่วยความจำก่อน ซึ่งอาจเป็นการโหลดตัวโปรแกรมจากเซิร์ฟเวอร์ หรือ พีซี โดยผ่านสายเคเบิลหรือเครือข่ายไร้สาย

2. Installation

หลังจากที่โหลดตัวแอปพลิเคชัน (MIDlet) เข้าสู่หน่วยความจำแล้ว ซอฟต์แวร์จัดการแอปพลิเคชันจะทำหน้าที่ตรวจสอบ (Preverified) MIDlet ว่ามีความถูกต้องและเหมาะสมกับอุปกรณ์หรือไม่

พร้อมกับการติดตั้งเพื่อที่จะให้ MIDlet พร้อมที่จะทำงานต่อไป ซึ่ง MIDlet จะอยู่ในตัวเครื่องจนกว่าจะมีการสั่งลบออกจากตัวเครื่อง

3. Launching (Run)

ขั้นตอนนี้จะเหมือนกับทำงานแอปพลิเคชันทั่วไป คือ ซอฟต์แวร์จัดการแอปพลิเคชันจะเรียก MIDlet ขึ้นมาทำงาน อาจจะเป็นการแสดงผลหน้าจออินพุต ให้ผู้ใช้กรอกข้อมูลสำหรับการทำงานใน ขั้นตอนนี้ MIDlet จะมีอยู่ 3 สถานะ ดังนี้

1. Pause state คือสถานะพร้อมที่จะทำงานกำลังรอรับคำสั่ง
2. Active state เป็นสถานะ การทำงานตามปกติของ MIDlet เมื่อได้รับคำสั่งและไม่มี Error Exception
3. Destroy state เป็นสถานะที่ MIDlet ปลดปล่อยหรือคืนทรัพยากรให้กับระบบเมื่อผู้ใช้ เลิกใช้ ปิดโปรแกรม หรือ เกิด Error Exception ขึ้น

4. Version Management

ซอฟต์แวร์จัดการแอปพลิเคชันจะทำหน้าที่คอยตรวจสอบเวอร์ชันของ MIDlet และคอยอัปเดต MIDlet ให้เป็นเวอร์ชันใหม่

5. Removal

เมื่อผู้ใช้ไม่มีความจำเป็นที่จะต้องใช้แอปพลิเคชัน ผู้ใช้สามารถลบ MIDlet นั้นออกจากเครื่องได้ เหมือนกับการลบแอปพลิเคชันทั่วไป

แอปพลิเคชันที่เขียนขึ้นมาเพื่อใช้กับอุปกรณ์ที่ใช้ MIDP เราจะเรียกว่า MIDlet แม้ว่าตัว MIDlet จะไม่มีเมธอด main() แต่ MIDlet ได้นำคลาส javax.microedition.midlet.MIDlet ตลอดจนเมธอดต่อไปนี้เข้ามาใช้งาน คือ

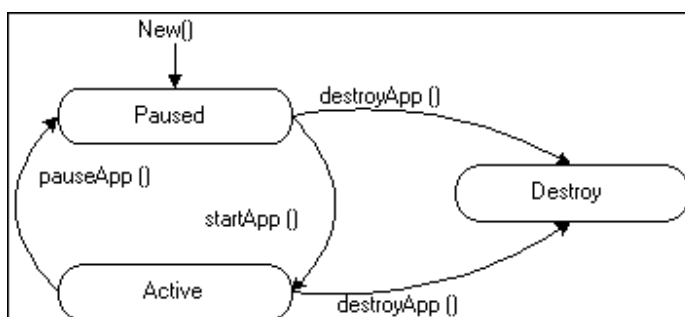
- startApp() ใช้สำหรับ allocate system resources และ กำหนดค่าเริ่มต้นของแอปพลิเคชัน (initialize application)
- pauseApp() ใช้หยุดการทำงานของ MIDlet ชั่วคราว โดยจะเปลี่ยนจาก Pause State ไปเป็น Start State โดยใช้คำสั่ง resumeRequest()
- destroyApp() ใช้ปล่อยการใช้งานทรัพยากรซึ่งถูกใช้โดย MIDlet และเพื่อหยุดการทำงานของ MIDlet

2.3.1.1 ขั้นตอนการทำงานของ MIDlet

การทำงานของ MIDlet ประกอบไปด้วย 3 สถานะ คือ กำลังทำงาน , หยุดชั่วคราว และถูกทำลาย โดยอาศัยซอฟต์แวร์จัดการแอปพลิเคชันเป็นตัวควบคุมการเปลี่ยนสถานะหนึ่งไปอีกสถานะหนึ่งด้วยเมธอด startApp() , pauseApp() และ destroyApp() ที่มาพร้อมกับ MIDlet ดังรูปที่ 2-6

เริ่มแรกซอฟต์แวร์จัดการแอปพลิเคชันจะสร้าง MIDlet ขึ้นมาก่อนโดย MIDlet ที่ถูกสร้างขึ้นมานั้นจะอยู่ในสถานะหยุดชั่วคราว จากนั้นซอฟต์แวร์จัดการแอปพลิเคชันจะเรียกเมธอด startApp() ขึ้นมาและ MIDlet จะเข้าสู่สถานะกำลังทำงาน ซึ่งจะเปิดรับทรัพยากรที่ต้องการและเริ่มต้นการทำงาน ในสถานะนี้ MIDlet จะดึงทรัพยากรที่ต้องการใช้งานเอาไว้ เมื่อซอฟต์แวร์จัดการแอปพลิเคชันต้องการให้ MIDlet หยุดทำงานชั่วคราวจะเรียกใช้เมธอด pauseApp() MIDlet จะหยุดทำงานและเข้าสู่สถานะหยุดชั่วคราวและคืนทรัพยากรที่ดึงมาใช้งาน MIDlet จะกลับไปอยู่สถานะกำลังทำงาน ได้เมื่อซอฟต์แวร์จัดการแอปพลิเคชันเรียกเมธอด resumeRequest()

สุดท้ายเมื่อซอฟต์แวร์จัดการแอปพลิเคชันไม่ต้องการเรียกใช้งาน MIDlet หรือต้องการคืนหน่วยความจำเพื่อให้โปรแกรมอื่นได้ใช้งาน ซอฟต์แวร์จัดการแอปพลิเคชันจะทำการทำลาย MIDlet ทิ้ง โดยเรียกเมธอด destroyApp() MIDlet จะเข้าสู่สถานะถูกทำลายและจะคืนทรัพยากรทั้งหมด ทำการจัดเก็บข้อมูลถาวรต่างๆ และหยุดการทำงานทั้งหมด



รูปที่ 2-6 ขั้นตอนการทำงานของ MIDlet

ตัวอย่างโครงสร้างแอปพลิเคชันของ MIDlet

```

import javax.microedition.midlet.*;
import javax.microedition.lcui.*;

class ClassName extends MIDlet implements CommandListener{

    // data or variable here

    //constructor
  
```

```

public ClassName(){
    // statement here
}

public void startApp(){
    // statement here
}

public void pauseApp(){
    //statement here
}

public void destroyApp(boolean unconditional){
    //statement here
}

public void commandAction(Command c, Displayable d){
    //statement here
}
}

```

คลาสที่เกี่ยวข้องกับ MIDlet

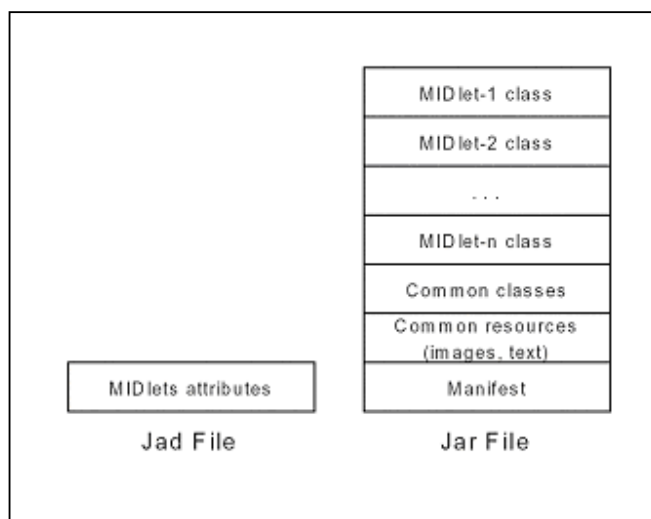
- javax.microedition.midlet เป็นคลาสที่ใช้ในการจัดการ Lifecycle ของ MIDlet
- javax.microedition.lcdui เป็นคลาสที่ใช้จัดการเกี่ยวกับการแสดงผลทางหน้าจอของอุปกรณ์
- javax.microedition.rms เป็นคลาสที่ใช้จัดการเกี่ยวกับการเก็บข้อมูลแบบคงตัว
- javax.microedition.io เป็นคลาสสำหรับติดต่อกับเครือข่าย

คลาสพื้นฐาน ที่มาจาก J2SE คือ java.io , java.lang และ java.util ซึ่ง class เหล่านี้จะเหมือนกับ class ของ J2SE แต่จะถูกตัดออกไปบางส่วน

2.3.1.2 MIDlet suite

MIDlet แอปพลิเคชันที่ถูกสร้างขึ้นจะต้องเก็บอยู่ในไฟล์ .jar เพียงไฟล์เดียวโดยนำมาทำการ Packaging รวมกันเป็นไฟล์ .jar ทั้งนี้ไฟล์ .jar ไฟล์หนึ่งอาจจะมีแอปพลิเคชัน MIDlet มากกว่าหนึ่งตัว ซึ่งเราเรียกกลุ่มของแอปพลิเคชัน MIDlet นี้ว่า “MIDlet Suite” ตามรูปที่ 2-8 หลังจากทำ Packaging แล้วสามารถ

ดาวน์โหลด และติดตั้งลงในอุปกรณ์ได้โดยอาจจะผ่านทางสายเคเบิลอนุกรมต่อเข้าโดยตรงจากคอมพิวเตอร์ หรือผ่านเครือข่ายไร้สาย Wireless network (OTA : Over-The-Air)



รูปที่ 2-7 MIDlet Suite

2.3.1.2.1 Manifest file

Manifest file เป็นไฟล์หนึ่งที่จะต้องถูกรวมอยู่ในไฟล์ .jar ของ MIDlet Suite มีไว้เพื่ออธิบายเนื้อหาในไฟล์ .jar และข้อมูลอื่นๆ เช่น ชื่อ , เวอร์ชัน , ผู้ผลิต MIDlet suite เป็นต้น Manifest file จะช่วยให้ผู้ใช้งานทราบว่า MIDlet ใดบ้างใน MIDlet suite ตารางที่ 2-2 แสดงแอททริบิวต์ต่างๆของ manifest file เนื่องจากจะมีเพียงบางแอททริบิวต์เท่านั้นที่จำเป็นต้องมี ส่วนแอททริบิวต์ที่เหลือจะมีหรือไม่ก็ได้

ชื่อแอททริบิวต์	รายละเอียด
MIDlet-Name	ชื่อของ MIDlet suite
MIDlet-Version	เวอร์ชันของ MIDlet suite
MIDlet-Vendor	ผู้ผลิต MIDlet suite
MIDlet-Icon	ชื่อไฟล์ PNG ในไฟล์ .jar จะใช้เมื่อซอฟต์แวร์จัดการแอปพลิเคชันแสดง ไอคอนเพื่อระบุโปรแกรมบนหน้าจอ
MIDlet-Description	คำอธิบายรายละเอียดของ MIDlet suite
MIDlet-Info-URL	URL สำหรับค้นหาคำอธิบายข้อมูลของ MIDlet suite เพิ่มเติม
MIDlet-<n>	ชื่อไอคอนและคลาสของ MIDlet ลำดับที่ n ในไฟล์ .jar กันด้วย

	เครื่องหมาย”,” โดยชื่อ ใช้เพื่อจำแนก MIDlet ให้ผู้ใช้งานทราบ ไอคอน เป็นชื่อไฟล์ที่ใช้แสดงเป็นรูปไอคอน
MIDlet-Jar-URL	URL ที่สามารถดาวน์โหลดไฟล์ .jar ได้
MIDlet-Jar-Size	ขนาดของไฟล์ .jar มีขนาดเป็นไบต์
MIDlet-Data-Size	ขนาดขั้นต่ำของพื้นที่เก็บข้อมูลแบบถาวรที่ MIDlet ใช้ มีหน่วยเป็น ไบต์ ค่าดีฟอลต์คือ 0
MicroEdition-Profile	โปรไฟล์ของ J2ME ที่ต้องการ
MicroEdition-Configuration	คอนฟิกูเรชันของ J2ME ที่ต้องการ

ตารางที่ 2-2 แสดงแอททริบิวต์ต่างๆของ manifest file

ตัวอย่างไฟล์ manifest file (MANIFEST.MF)

```

MIDlet-Name: HelloMidlet
MIDlet-Vendor: Olala
MIDlet-Version: 1.0
MIDlet-1: HelloMidlet, /Icon.png, HelloMidlet
MicroEdition-Configuration: CLDC-1.0
MicroEdition-Profile: MIDP-1.0
MIDlet-Data-Size: 0

```

2.3.1.2.2 Application Descriptor

ไฟล์ application descriptor จะมีรูปแบบเหมือนกับไฟล์ manifest file แต่มีวัตถุประสงค์ในการใช้งานที่แตกต่างกัน และจะไม่อยู่ในไฟล์ .jar เหมือน manifest file แต่จะแยกเป็นไฟล์ต่างหากโดยใช้นามสกุล .jad ไฟล์ application descriptor จะใช้ในกรณีที่ต้องการดาวน์โหลดแอปพลิเคชันแบบไร้สาย โดยก่อนที่จะดาวน์โหลดไฟล์ .jar ลงอุปกรณ์ ซอฟต์แวร์จัดการแอปพลิเคชันจะอ่านไฟล์ application descriptor เพื่อตรวจสอบก่อนว่า MIDlet suite นั้น สามารถใช้ได้กับอุปกรณ์ที่จะติดตั้งหรือไม่ ซึ่งจะมีประโยชน์มากในกรณีที่เครือข่ายไร้สายมีแบนด์วิดท์ต่ำเพราะหาก MIDlet ไม่เหมาะสมกับอุปกรณ์ เช่นแอปพลิเคชันมีขนาดใหญ่เกินไป , แอปพลิเคชันใช้คอนฟิกูเรชัน หรือโปรไฟล์ที่ไม่เหมาะสมกับอุปกรณ์ เป็นต้น ซอฟต์แวร์จัดการแอปพลิเคชันจะไม่ทำการดาวน์โหลดแอปพลิเคชันดังกล่าว

เนื่องจากไฟล์ application descriptor เป็นไฟล์ที่ใช้ก่อนติดตั้งแอปพลิเคชัน ดังนั้นจึงมีแอททริบิวต์บังคับอยู่ 7 แอททริบิวต์ ที่ต้องอยู่ในไฟล์ application descriptor เพื่อใช้ในการตรวจสอบ คือ

- MIDlet-Name
- MIDlet-Version
- MIDlet-Vendor
- MIDlet-Jar-URL
- MIDlet-Jar-Size
- MicroEdition-Profile
- MicroEdition-Configuration

2.3.1.2.3 องค์ประกอบของ MIDlet

MIDlet เมื่อถูกสร้างขึ้นจะต้องนำมารวมกันเป็นแพ็คเกจ ผลจากการรวมเรียกว่า MIDlet Suite ซึ่ง จะประกอบด้วยไฟล์ 2 ไฟล์ ต่อไปนี้

1) JAR file

ประกอบด้วยคลาสไฟล์ของ MIDlet ที่ทำการคอมไพล์และตรวจสอบแล้ว และไฟล์ทรัพยากรต่างๆ เช่น รูปภาพ , ไฟล์เสียง เป็นต้น โดยปกติไฟล์ .jar ของ MIDlet Suite จะประกอบด้วยคลาสไฟล์ทั้งหมดของ MIDlet แอปพลิเคชันรวมถึงไฟล์ทรัพยากร และ Manifest file โดยที่คลาสไฟล์จะต้องถูกทำการตรวจสอบก่อน ส่วนไฟล์ทรัพยากรจะประกอบด้วย ตัวอักษร, ไฟล์รูป ที่ถูกใช้โดย MIDlet ตอนรันไทม์ แต่ละ MIDlet ที่อยู่ใน MIDlet Suite เดียวกันสามารถใช้คลาสไฟล์ร่วมกันได้ ถ้าเราสร้าง MIDlet หลายๆ ตัว โดยทำเป็นแพ็คเกจเดียวกันก็จะประหยัดขนาดแพ็คเกจที่ต้องดาวน์โหลดได้มาก เพราะสามารถใช้วิธีการใช้คลาสไฟล์ร่วมกันได้

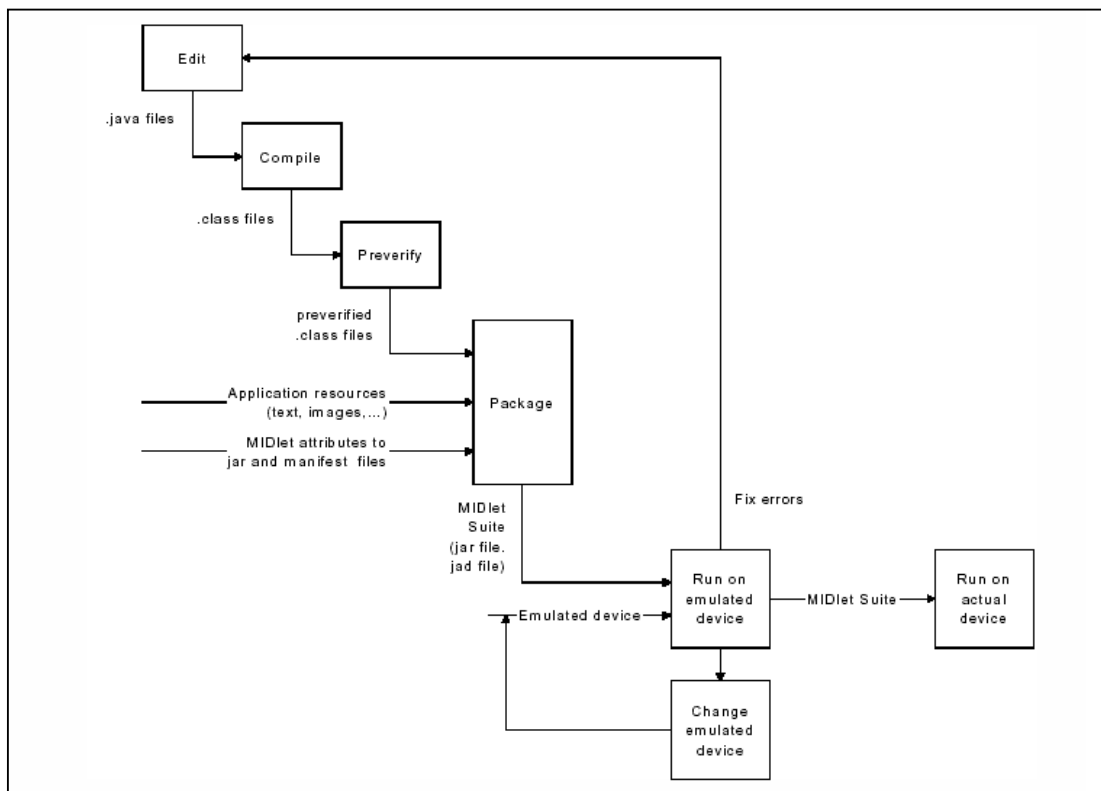
2) JAD file

เป็นไฟล์ที่ทำหน้าที่เป็น "Application Descriptor" ลักษณะของไฟล์จะคล้ายกับ Manifest file ที่อยู่ในไฟล์ .jar แต่ใช้คนละจุดประสงค์กัน โดย Manifest file จะใช้สำหรับ packaging MIDlet Suite แต่ไฟล์ .jad จะใช้ตอนดาวน์โหลด MIDlet ลงอุปกรณ์แบบไร้สาย

2.3.1.2.4 ขั้นตอนการพัฒนา MIDlet

1. เขียนโปรแกรม โดยโปรแกรมที่ถูกพัฒนาขึ้นจะมีนามสกุลเป็น .java
2. คอมไพล์โปรแกรมหinggaได้ไฟล์ .class
3. รันคลาสที่ได้จากการคอมไพล์ผ่าน preverify tool เป็นการตรวจสอบความถูกต้อง (preprocess) และสร้างคลาสสำหรับรันใน KVM โดยจะไม่เปลี่ยนชื่อคลาสแต่จะเพิ่มคุณสมบัติพิเศษที่ใช้รันใน KVM

4. หลังจากทดสอบ MIDlet แล้วเราจำเป็นต้องรวม MIDlet ทุกตัวเข้าเป็นไฟล์เดียวกัน (นามสกุลเป็น .jar) เรียกขั้นตอนนี้ว่า “Packaging” และเรียกกลุ่มของ MIDlet ที่ได้ว่า “MIDlet Suite” หลังจากที่ได้ MIDlet Suite แล้ว ต่อไปเป็นการสร้าง manifest file และไฟล์ Application descriptor ที่มีนามสกุลเป็น .jad ซึ่ง manifest file จะเก็บข้อมูลทั้งหมดของตัว MIDlet เช่น ชื่อ , เวอร์ชัน , ผู้สร้าง , ที่เก็บตัว MIDlet , ขนาดของ MIDlet , เวอร์ชันของโปรแกรมไฟล์และคอนฟิกูเรชัน และรายชื่อไฟล์ที่อยู่ใน MIDlet Suite (.jar)ว่ามี MIDlet ชื่ออะไรบ้าง ส่วนไฟล์ Application descriptor นำมาใช้เพื่อบอกตำแหน่งของไฟล์ .jar ข้อมูลเหล่านี้จะถูกเรียกใช้โดยซอฟต์แวร์จัดการแอปพลิเคชันหรือบางครั้งจะถูกเรียกใช้โดยตัว MIDlet เอง
5. เริ่มทดสอบ MIDlet โดยใช้อิมูเลเตอร์ทดสอบโปรแกรม หากเจอข้อผิดพลาดระหว่างการทดสอบสามารถทำการแก้ไขโปรแกรมใหม่ได้ โดยกลับไปทำตั้งแต่ขั้นตอนแรกอีกครั้ง และขณะทำการทดสอบแอปพลิเคชันบนอิมูเลเตอร์ สามารถเปลี่ยนรูปแบบของอิมูเลเตอร์ที่นำมาทดสอบได้
6. หากไม่พบข้อผิดพลาดใดๆ สามารถทำการดาวน์โหลดแอปพลิเคชันลงในอุปกรณ์ได้
ขั้นตอนในการพัฒนาแอปพลิเคชันมีขั้นตอนต่างๆ ดังรูปที่ 2-7



รูปที่ 2-8 ขั้นตอนการพัฒนา MIDlet

2.3.2 โครงสร้างส่วนติดต่อกับผู้ใช้ของ MIDP

APIs สำหรับส่วนติดต่อกับผู้ใช้ของ MIDP ถูกออกแบบขึ้นมาใหม่โดยไม่อ้างอิงซบเซตของ AWT (Abstract Window Toolkit) ซึ่งออกแบบมาสำหรับเครื่องคอมพิวเตอร์พีซีที่มีอุปกรณ์การชี้ (Pointer Device) จอภาพขนาดใหญ่พร้อมกับหน่วยความจำที่เพียงพอ ในขณะที่หน้าจอแสดงผลของอุปกรณ์ไร้สายมีขนาดเล็กกว่า และใช้เป็นกคเพื่อป้อนข้อมูลเข้าแทนที่จะเป็นตัวชี้ โดย APIs สำหรับส่วนติดต่อกับผู้ใช้ของ MIDP จะกำหนดอยู่ในแพ็คเกจ javax.microedition.lcdui

APIs ของส่วนติดต่อกับผู้ใช้ใน MIDP มี 2 กลุ่มหลักๆ คือ APIs ระดับสูง (high-level APIs) และ APIs ระดับต่ำ (low-level APIs) แสดงดังรูปที่ 2-9

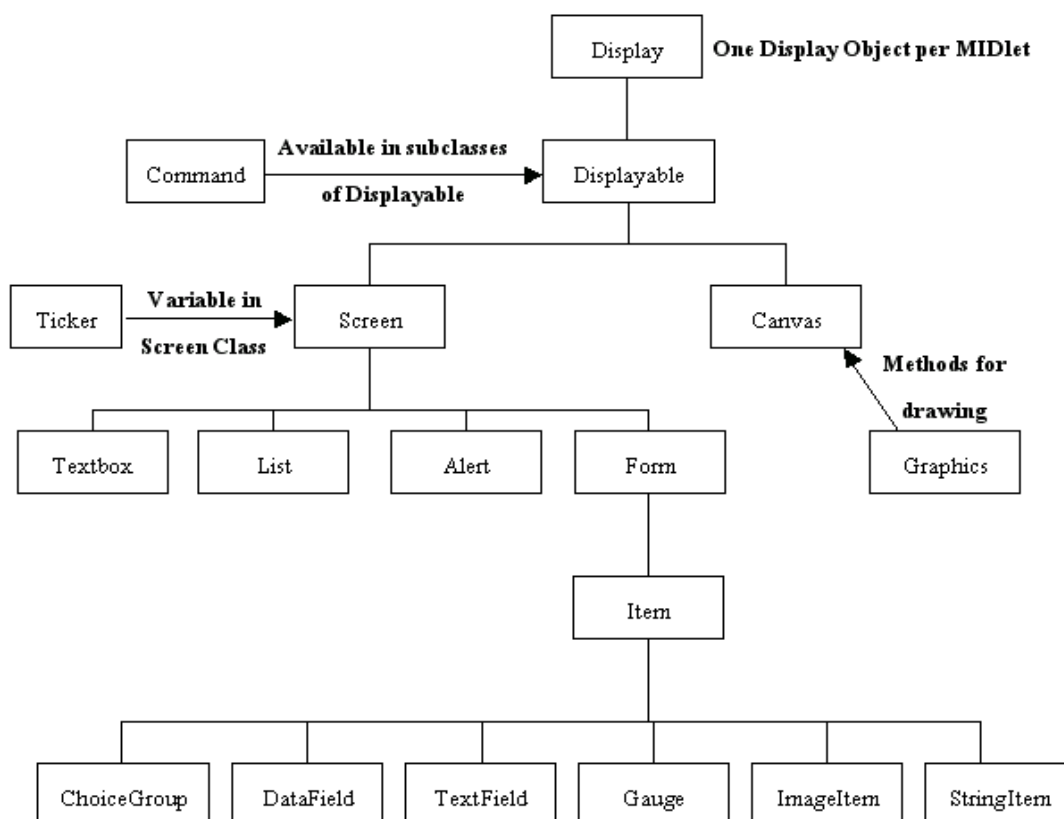
APIs ระดับสูงออกแบบให้เป็น abstract เน้นความยืดหยุ่นในการโยกย้ายไปใช้งานในระบบอื่นๆ แอปพลิเคชันที่ใช้ APIs ระดับสูงจะควบคุมวิธีการแสดงผลได้น้อยมากและเข้าถึงได้เฉพาะอีเวนต์ระดับสูงเท่านั้น ซึ่ง APIs ระดับสูงนี้ถูกอิมพลิเมนต์โดยคลาสที่สืบทอดมาจากคลาส Screen

APIs ระดับต่ำจะสามารถควบคุมการแสดงผลได้เต็มที่ สามารถเข้าถึงอุปกรณ์ป้อนข้อมูลและจัดการอีเวนต์พื้นฐานที่เกิดจากการโต้ตอบของผู้ใช้ได้ ซึ่งถูกอิมพลิเมนต์โดยคลาสที่สืบทอดมาจากคลาส Canvas แต่แอปพลิเคชันที่ใช้ APIs ระดับต่ำนี้จะขาดความยืดหยุ่น ยึดติดกับอุปกรณ์มากจนเกินไป แอปพลิเคชันต้องมีการกำหนดตำแหน่งที่แน่นอน และต้องระวังเรื่องความแตกต่างของหน้าจอแสดงผล แต่หากเป็นแอปพลิเคชันที่ใช้ APIs ระดับสูงจะไม่ต้องระวังเรื่องความแตกต่างของขนาดหน้าจอมากนัก เนื่องจาก APIs ระดับสูงจะจัดการกับความแตกต่างของหน้าจอให้ด้วย

ในการแสดงผลข้อมูลบนหน้าจอใน J2ME MIDP ออบเจกต์ Displayable จะเก็บข้อมูลที่ต้องการ นำเสนอ และ ออบเจกต์ Display จะนำออบเจกต์ Displayable นั้นไปแสดงบนจอให้ผู้ใช้เห็น ในโปรแกรม MIDlet หนึ่งๆ จะมีออบเจกต์ของ Display ได้เพียงตัวเดียวเท่านั้น แอปพลิเคชันสามารถอ้างถึงออบเจกต์ Display ได้ด้วยเมธอด getDisplay() และสามารถเรียกเมธอดนี้ได้ทุกเมื่อตั้งแต่เริ่มเรียกใช้ startApp() จนถึง destroyApp() ออบเจกต์ Display ทั้งหมดที่ได้จาก getDisplay() จะอยู่จนกว่าโปรแกรมจะทำงานเสร็จ

ในส่วนของคลาส Displayable เป็นคลาสหลักของส่วนติดต่อกับผู้ใช้ของ MIDP สามารถเก็บภาพกราฟิกที่สร้างเฉพาะอุปกรณ์และนำไปแสดงผลที่หน้าจอ ซึ่งประกอบไปด้วย 3 กลุ่มย่อยๆ คือ

- Structure - predefined Screen (Alert , List , Textbox) แอปพลิเคชันจะไม่สามารถเพิ่ม คอมโพเนนต์อื่นๆลงในออบเจกต์นี้ได้
- Screen ทั่วไป (Form) แอปพลิเคชันสามารถเพิ่มคอมโพเนนต์ลงในออบเจกต์ของคลาสนี้ได้ เช่น ข้อความ , รูป
- Canvas แอปพลิเคชันสามารถควบคุมการแสดงผลของคอมโพเนนต์ได้ทั้งหมด และสามารถเข้าถึงอีเวนต์ระดับล่างได้



รูปที่ 2-9 โครงสร้างระดับชั้นของคลาส Display

2.3.2.1 APIs ระดับล่าง

APIs ระดับล่างประกอบด้วย 2 ส่วน คือ

1. คลาส Canvas ซึ่งมีเมธอด paint() สำหรับการวาด และเป็นคลาสที่สร้างและจัดการอีเวนต์ระดับล่างเช่น การกดปุ่ม เมธอด paint() ถูกประกาศเป็น abstract เช่นเดียวกับคลาส Canvas ดังนั้นแอปพลิเคชันต้องขยายคลาส Canvas เพื่อใช้งานและจะต้องมีเมธอด paint() แต่ไม่จำเป็นต้องมีเมธอดจัดการอีเวนต์ครบ ทั้งหมด จะมีเฉพาะที่แอปพลิเคชันสนใจเท่านั้น คลาสใดที่ต้องการจัดการอีเวนต์ระดับล่างและเรียกใช้ เมธอดทางกราฟิกเพื่อวาดภาพบนหน้าจอจะต้องอิมพลิเมนต์คลาส Canvas ดังรูปที่ 2-10

จุดประสงค์ในการใช้คลาส Canvas คือ เพื่อเข้าถึงและจัดการอีเวนต์ระดับล่าง แต่อีเวนต์ระดับสูงสามารถเกิดจาก Canvas ได้ แอปพลิเคชันสามารถเพิ่มคำสั่งใน Canvas ได้โดยใช้เมธอด addcommand() เหมือนกับซับคลาสของ Displayable() อื่นๆ และยังสามารถรีจิสเตอร์ CommandListener ใน Canvas ด้วยเมธอด setCommandListener() ได้เช่นกัน

2. คลาส Graphics ทำหน้าที่สร้างภาพเรขาคณิต 2 มิติ คลาสนี้คล้ายกับ java.awt.Graphics ใน J2SE ซึ่งจะมีเมธอดไว้สำหรับวาดข้อความ , รูป , เส้น , วาด/เติมรูปเหลี่ยม และวาดส่วนโค้งบนจอ

2.3.2.2 APIs ระดับสูง

คลาส Screen เป็นซับคลาสของ Displayable ซึ่งนำ APIs ระดับสูงมาใช้ ซับคลาสของ Screen แบ่งออกเป็น 2 กลุ่ม คือ

1. ประกอบด้วยคลาส Alert , List และ Textbox ซึ่งมีโครงสร้างที่กำหนดไว้ล่วงหน้า
2. คลาส Form ซึ่งกำหนดโครงสร้างโดยแอปพลิเคชัน ใน Form สามารถบรรจุ Item ต่างๆ เช่น ภาพ , ข้อความ หรือ choice ได้อย่างไม่จำกัดจำนวน แต่ Item หนึ่งจะอยู่ได้ใน Form เดียวเท่านั้น ดังนั้น หากพยายามใส่ Item ที่มีอยู่แล้วลงใน Form อื่นๆ จะเกิด IllegalStateException ขึ้น ดังนั้นจะต้องย้าย Item ออกจาก Form ที่อยู่เดิมเสียก่อนจึงจะสามารถใส่ Form ใหม่เข้าไปได้

คลาส Item เป็นซูเปอร์คลาสสำหรับคอมโพเนนต์ต่างๆ ที่สามารถใส่ลงใน Form ได้ โครงสร้างลำดับชั้นของ Item แสดงดังรูปที่ 2-9 ซึ่งคลาส Item เป็นออกเป็น 2 กลุ่ม

- ChoiceGroup , Gauge , DateField , TextField เป็น Item ที่ผู้ใช้งานสามารถแก้ไขค่าของ คอมโพเนนต์ได้
- StringItem , ImageItem เป็น Item ที่ผู้ใช้ไม่สามารถแก้ไขค่าได้ จะต้องทำผ่านแอปพลิเคชันเท่านั้น

```
class TestCanvas extends Canvas{
...
    void paint(Graphics g){
        ...
    }
...
}
```

รูปที่ 2-10 แสดงการอิมพลิเมนต์คลาส Canvas

2.4 กรอบการติดต่อสื่อสารทั่วไป (The Generic Connection Framework - GCF)

ใน J2SE เราสามารถพัฒนาแอปพลิเคชันสำหรับเครือข่ายได้ง่าย เพราะ J2SE ได้ให้คลาสไลบรารีของเครือข่ายซึ่งมีฟังก์ชันให้ใช้มากมาย ทั้งในแพ็คเกจ `java.io` มีคลาสที่สนับสนุนการส่งข้อมูล และในแพ็คเกจ `java.net` ที่เป็นคลาสสนับสนุนการทำงานด้านเครือข่าย แต่คลาสทั้งสองออกแบบมาสำหรับระบบคอมพิวเตอร์ทั่วไปที่มีหน่วยประมวลผล, หน่วยความจำ และพื้นที่เก็บข้อมูลที่เพียงพอ ไฟล์คลาสเหล่านี้มีขนาดประมาณ 200 กิโลไบต์ ซึ่งมีขนาดใหญ่เกินกว่าจะส่งไปในอุปกรณ์ไร้สาย ที่มีทรัพยากรต่างๆ จำกัด

ดังนั้นจึงได้มีการพัฒนากรอบการติดต่อสื่อสารทั่วไปเข้ามาใช้ใน CLDC แนวคิดของกรอบการติดต่อสื่อสารทั่วไป คือ การกำหนด abstraction ของเครือข่ายและไฟล์ I/O เพื่อสนับสนุนอุปกรณ์แบบพกพาประเภทต่างๆ ไว้ในอินเตอร์เฟสของจาวา และให้ผู้ผลิตอุปกรณ์แต่ละรายนำ abstract ที่กำหนดมาใช้งานจริง

ในการติดต่อสื่อสารนั้น มีคลาส `Connector` เป็นคลาสหลักในการติดต่อ ไม่ว่าจะเป็นการเชื่อมต่อโดยไฟล์ I/O การสื่อสารผ่านพอร์ตอนุกรม การเชื่อมต่อแบบดาต้าแกรม หรือแบบ Http จะต้องสร้างการเชื่อมต่อจากเมธอด `open()` ในคลาส `Connector` เหมือนกัน แสดงดังรูปที่ 2-11 จากคุณลักษณะดังกล่าวทำให้ J2ME มีความยืดหยุ่นและสามารถรองรับอุปกรณ์หรือผลิตภัณฑ์ใหม่ๆ ได้มากขึ้น เมธอด `open` มีลักษณะดังนี้

`Connection open(String connect_string)`

`Connect_string` ใช้รูปแบบของ `{protocol}:{target} [{parameters}]` ซึ่งคล้ายคลึงกับรูปแบบของ URL ที่ใช้ทั่วกันทั่วไป เช่น <http://www.kmitl.ac.th/> โดยประกอบไปด้วย 3 ส่วนคือ

1. โพรโตคอลจะกำหนดชนิดของการเชื่อมต่อที่สร้างโดยเมธอด `open()` มีค่าเป็นไปได้หลายค่าดังนี้
 - File ไฟล์ I/O
 - Comm การสื่อสารผ่านพอร์ตอนุกรม
 - Socket การสื่อสารแบบซ็อกเก็ต ที่ใช้โปรโตคอล TCP/IP
 - Datagram การสื่อสารแบบดาต้าแกรม
 - http การเข้าถึงเว็บเซิร์ฟเวอร์
2. ปลายทางอาจเป็นชื่อคอมพิวเตอร์, หมายเลขพอร์ต, ชื่อไฟล์ หรือหมายเลขพอร์ตติดต่อสื่อสารก็ได้
3. พารามิเตอร์ เป็นตัวเลือกที่มีหรือไม่มีก็ได้ โดยจะระบุข้อมูลเพิ่มเติมที่จำเป็นต้องใช้เพื่อให้สตรีมการเชื่อมต่อสมบูรณ์

คลาสและอินเตอร์เฟสทั้งหมดของ J2ME รวมอยู่ในแพ็คเกจเดียว คือ `javax.microedition.io` และเพื่อให้การติดต่อสื่อสารมีความยืดหยุ่นและสนับสนุนอุปกรณ์ได้หลากหลายตามที่ต้องการ จึงได้นำการติดต่อสื่อสารรูปแบบต่างๆ มาใช้ ซึ่งมีรูปแบบการติดต่อ 7 ลักษณะ คือ

■ Connection

- ContentConnection
- DatagramConnection
- InputConnection
- OutputConnection
- StreamConnection
- StreamConnectionNotifier

HTTP:

```
Connector.open("http://www.kmitl.ac.th");
```

Sockets:

```
Connector.open("socket://someaddress:80");
```

Datagrams:

```
Connector.open("datagram://www.kmitl.ac.th");
```

Serial Port:

```
Connector.open("comm;0;baurate=0000");
```

File:

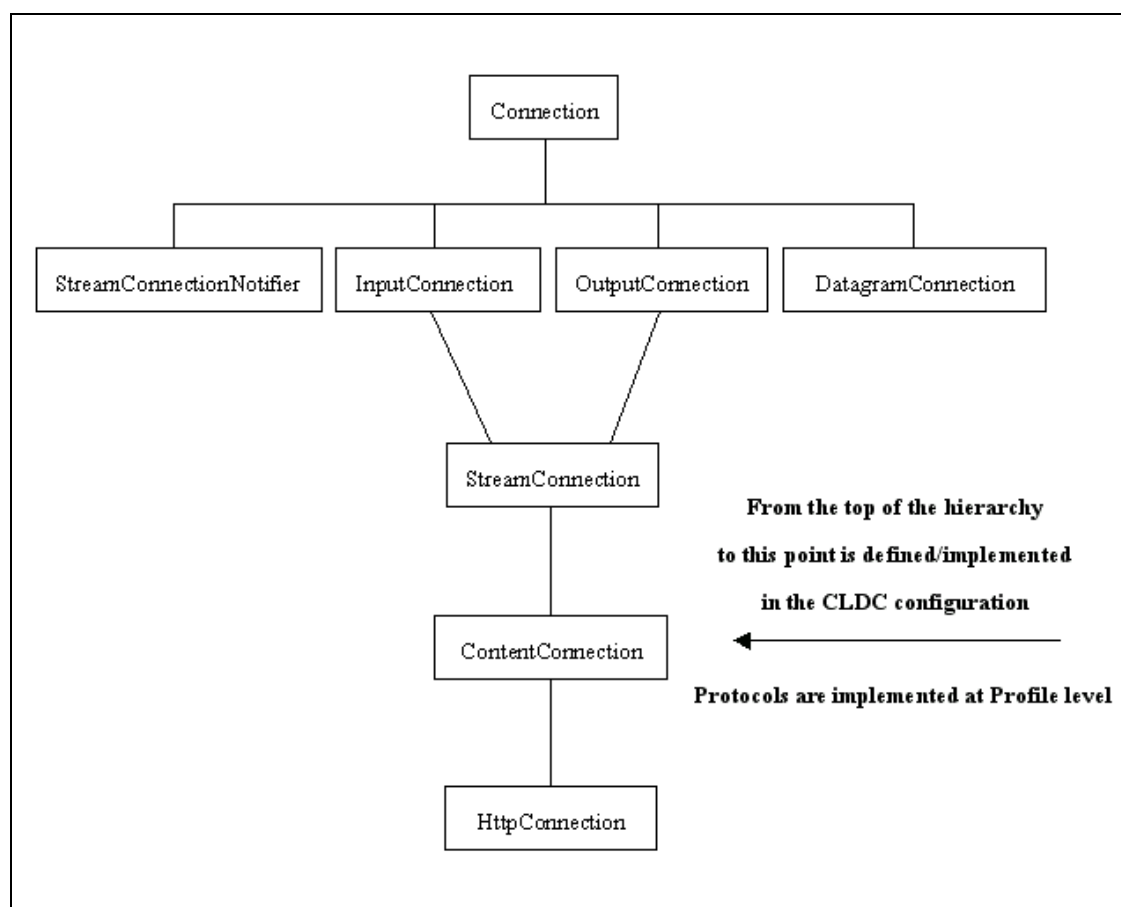
```
Connector.open("file:/testdata.dat");
```

รูปที่ 2-11 รูปแบบพื้นฐานในการติดต่อสื่อสาร

Connection เป็นอินเทอร์เฟซพื้นฐานที่เป็นจุดเริ่มต้นของอินเทอร์เฟซการเชื่อมต่อชั้นต่าง ๆ อินเทอร์เฟซการเชื่อมต่ออื่นๆ ล้วนมาจาก Connection.StreamConnection ซึ่งเกิดจาก InputConnection และ OutputConnection โดยกำหนดความสามารถในการรับข้อมูลเข้าและส่งออกเมื่อมีการเชื่อมต่อเครือข่าย ContentConnection มีที่มาจาก StreamConnection โดยเพิ่มเมธอดเข้าไปสำหรับการจัดการข้อมูลแบบ MIME HttpConnection มีที่มาจาก ContentConnection จากรูปที่ 2-12 แสดงถึงโครงสร้างของการติดต่อ ซึ่งคลาส Connector และ อินเทอร์เฟซ Connection ถูกกำหนดให้อยู่ใน CLDC ส่วนการอิมพลีเมนต์ โปรโตคอลจะอยู่

ในชั้นโพรไฟล์ ในปฏิญานพันธฉบับนี้จะเน้นไปที่คลาส `HttpConnection` ที่สนับสนุน HTTP เวอร์ชัน 1.1 ซึ่งเป็นโปรโตคอลที่ MIDP 1.0 ต้องการ

`HttpConnection` เป็นอินเทอร์เฟซบังคับที่ต้องมีใน MIDP ของผู้ผลิตทุกราย ดังนั้น `HttpConnection` จึงถูกระบุไว้ในข้อกำหนดของ MIDP โดยเน้นไปที่กลุ่มอุปกรณ์โทรศัพท์เคลื่อนที่และเพจเจอร์รับส่งข้อความ ซึ่งมีเมฆอดบรรจุไว้และสนับสนุนโปรโตคอล HTTP 1.1 ทำให้อุปกรณ์ที่นำคลาส อินเทอร์เน็ตมาใช้สามารถเชื่อมต่อในแบบ HTTP ได้



รูปที่ 2-12 ลำดับชั้นของอินเทอร์เน็ตการเชื่อมต่อ